

BUILD-SUPABASE.md

Partie C.8 — Vues PostgreSQL (Views)

Version : V1

Statut : Référence officielle des vues PostgreSQL

1. Objectif

Cette section définit les règles officielles d'utilisation des vues PostgreSQL dans Vitala.

Les vues permettent de :

- simplifier les requêtes complexes ;
- créer des projections de lecture ;
- améliorer la réutilisation des requêtes ;
- renforcer certains mécanismes de sécurité.

Les vues ne remplacent jamais les tables ni la logique métier.

2. Principes directeurs

Les vues doivent respecter les principes suivants :

- Read Model First
- Simplicité
- Réutilisabilité
- Performance
- Sécurité
- Documentation obligatoire

Une vue ne doit jamais devenir un substitut à une architecture correctement modélisée.

3. Types de vues autorisés

Vitala distingue deux catégories principales :

Vues classiques (VIEW)

Utilisées pour :

- simplifier les lectures ;
 - centraliser des jointures récurrentes ;
 - exposer des données consolidées.
-

Vues matérialisées (MATERIALIZED VIEW)

Utilisées pour :

- les agrégations coûteuses ;
- les statistiques ;
- les tableaux de bord ;
- les rapports.

Leur actualisation (REFRESH) doit être planifiée et documentée.

4. Cas d'utilisation

Les vues sont recommandées pour :

- les tableaux de bord ;
- les listes enrichies ;
- les rapports ;
- les projections destinées aux APIs ;
- les analyses des moteurs d'intelligence.

Chaque vue doit répondre à un besoin clairement identifié.

5. Cas à éviter

Les vues ne doivent pas être utilisées pour :

- implémenter des règles métier ;
 - remplacer les tables ;
 - masquer une mauvaise modélisation des données ;
 - effectuer des traitements complexes qui relèvent de la couche Domaine.
-

6. Nommage

Les vues doivent respecter les conventions suivantes :

- minuscules ;
- `snake_case` ;
- nom explicite.

Exemples :

- `vw_user_dashboard`
- `vw_active_missions`
- `vw_trust_summary`

Le préfixe `vw_` est recommandé afin de distinguer immédiatement une vue d'une table.

7. Sécurité

Les vues doivent respecter les politiques de sécurité de Vitala.

Elles ne doivent jamais exposer :

- des données sensibles non autorisées ;
- des colonnes confidentielles ;
- des informations contournant les politiques RLS.

Les permissions doivent être définies explicitement.

8. Performances

Les vues doivent être conçues pour limiter :

- les jointures inutiles ;
- les calculs coûteux ;
- les dépendances excessives.

Une vue fréquemment utilisée doit être évaluée afin de déterminer si une vue matérialisée serait plus appropriée.

9. Dépendances

Une vue peut dépendre :

- de tables ;

- d'autres vues.

Cependant, les chaînes de dépendances profondes doivent être évitées afin de préserver la lisibilité et les performances.

10. Documentation

Chaque vue doit être documentée avec :

- son objectif ;
 - les tables utilisées ;
 - les colonnes exposées ;
 - les utilisateurs ou services concernés ;
 - les règles de sécurité applicables.
-

11. Évolution

Toute création, modification ou suppression d'une vue doit :

- être réalisée via une migration ;
 - être testée ;
 - être documentée ;
 - préserver la compatibilité lorsque cela est possible.
-

12. Monitoring

Les vues critiques doivent être surveillées afin d'identifier :

- les requêtes lentes ;
- les dépendances cassées ;
- les impacts sur les performances.

Les vues matérialisées doivent également être contrôlées pour vérifier que leur actualisation reste conforme aux besoins métier.

13. Critères de conformité

Une vue est conforme lorsque :

- elle répond à un besoin identifié ;
- elle ne contient pas de logique métier ;
- elle respecte les conventions de nommage ;
- elle applique les règles de sécurité ;

- elle est documentée ;
 - ses performances sont validées.
-

14. Conclusion

Les vues PostgreSQL constituent un outil puissant pour simplifier les lectures et optimiser certaines requêtes dans Vitala.

Leur utilisation doit rester maîtrisée afin de préserver une architecture claire, performante et conforme aux principes du Build Blueprint.